claude-windows / df5d5fb2-534a-447a-8181-b222f94655f3

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\df5d5fb2-534a-447a-8181-b222f94655f3\subagents\workflows\wf_133acdee-dab\agent-aedb5e5a63d7bb30b.jsonl`
- SHA-256: `dcab22f96b14e4be6d4cf6d86c18267f5ca8125e26dfc8d69648da6cc7bec165`
- Source modified: `2026-07-06T17:32:58+00:00`
- Imported at: `2026-07-08T15:59:35+00:00`
- project: `wf_133acdee-dab`
- session_id: `df5d5fb2-534a-447a-8181-b222f94655f3`

Transcript

1. user (2026-07-06T17:31:53.589Z)

```
You are the SYNTHESIS lead for the wf294-fix review. Below are the review findings that
SURVIVED adversarial verification (CONFIRMED or UNCERTAIN). Dedupe overlapping findings, drop
any
that are actually fine on reflection, rank the rest most-severe-first, and give an overall
verdict
on whether this fix is safe to commit as-is or needs changes first. Be decisive and concrete;
distinguish "must fix before commit" from "nice-to-have follow-up". Do not invent new findings
not
grounded in the list below.


REPO ROOT: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer
DIFF UNDER REVIEW (read this first): C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-
Projects-badminton-highlight-
indexer/df5d5fb2-534a-447a-8181-b222f94655f3/scratchpad/wf294_fix.diff

GOAL OF THE CHANGE (wf294 incident): the Cloud Run rally-worker reported a run as
status="success" with 0 segments even when the WASB shuttle-trajectory detector TIMED OUT
(1800s) and produced zero trajectory. A detector timeout/abort was indistinguishable from a
legitimately 0-rally video. The fix must make "detector failed/timed out" surface as a
failed status (NOT "success") in report.json AND the completion marker, while a genuine
0-rally match stays a success.

WHAT THE FIX DID (4 files, all in the diff):
1. backend/pipeline/segmenters/trajectory_hybrid.py ? process_video() now returns a
   backend.results.Failure (instead of a bare []) on TWO detector-failure paths: the env
   healthcheck-not-ready path, and the "run_predict produced no trajectory" abort path.
   A trajectory-produced-but-zero-windows case still returns bare [] (a legit 0-rally).
   Return annotation widened to Union[List[Dict[str,Any]], Failure]; kept the
   # type: ignore[override].
2. backend/worker/__main__.py ? added _serialize_and_push_outputs() helper (writes+pushes
   intervals.csv + report.json). _fail(rc) now writes+pushes a status="failed" report.json /
   intervals.csv (0 segments), best-effort, BEFORE the M6 done-marker. The success path uses
   the same helper. The segmenter-Failure branch already routed to _fail(3).
3. tests/test_tracknet_hybrid.py ? the two abort tests now assert isinstance(res, Failure);
   added a test that zero-windows stays a bare [] (not a Failure).
4. tests/test_worker.py ? new test that a segmenter Failure -> rc 3 + report.json
   status="failed" + done-marker returncode 3/status="failed"; strengthened the 0-segment
   test to assert status="success".

KEY CONSUMER CODE TO CROSS-CHECK (read the real files, do not assume):
- backend/worker/__main__.py: cmd_infer (the full control flow: validation -> segmenter ->
  isinstance(result, Failure) -> _fail(3); the finally: emit_indexer_report; the success tail),
  _write_done_marker, _write_report_json, _write_intervals_csv.
```

- backend/orchestration.py: cached_process_result (~line 288, requires report status=="success" to hydrate), probe_process_cache (~line 255), _read_bucket_json, _resume_one_remote_job usage.
- backend/compute/cloud_run.py: CloudRunCompute.run_infer (reads the done-marker returncode + outputs/<id>/report.json).
- backend/api/job_worker.py: _resume_one_remote_job (waits for report.json to EXIST, then reads status; marks the job failed if status != success), _run_claimed_job.
- backend/results.py: Success/Failure/Result/unwrap_or_failure contract.
- backend/main.py (isinstance Failure branch ~1557), backend/pipeline/segmenters/gemini.py & motion.py (the existing Failure precedent), yolo_hybrid.py (a PARALLEL hybrid segmenter).

CONSTRAINTS THE FIX MUST RESPECT: ruff clean, mypy clean (trajectory_hybrid is NOT quarantined; worker.__main__ IS quarantined via mypy.ini ignore_errors), full suite green, coverage floor 70%.
The full suite is already green (1736 passed) and ruff/mypy clean ? so do NOT report "tests might fail" speculatively; report only concrete defects you can point at in the code.

Only report REAL defects: a wrong/edge behavior, a silent-failure path still leaking as success, a control-plane interaction regression, an idempotency/concurrency bug, a broken test assertion, or a genuine convention violation. Cite exact file:line. If a concern is speculative or you cannot ground it in the actual code, DO NOT report it.

SURVIVING FINDINGS (JSON):

```
[
  {
    "dimension": "completeness",
    "severity": "high",
    "verdict": "CONFIRMED",
    "file": "backend/pipeline/segmenters/trajectory_hybrid.py",
    "line": 219,
    "summary": "The missing-API-key abort still returns a bare [] instead of a Failure, so an unrunnable config surfaces as a 0-segment status=\"success\" ? the exact wf294 silent-failure class the fix set out to close.",
    "detail": "process_video has three pre-detection config-abort paths that STILL return bare []: unknown sport (line 194, KeyError), missing ${PROVIDER}_API_KEY (line 219), and unknown provider (line 226, KeyError). The missing-key case is the most acute: the worker's OWN 0-segment diagnostic warning in backend/worker/__main__.py:340-352 explicitly names it as a top cause of a false 0-segment 'success' ('a missing ${AI_PROVIDER}_API_KEY with skip_ai_handoff=False (the segmenter bails BEFORE detection)'). The sibling gemini segmenter already treats exactly these three conditions as a Failure ? backend/pipeline/segmenters/gemini.py:100/116/124, whose docstring cites 'Finding B ? the loud-failures bar ... not a silent 0-rally success that looks like an empty match'. motion.py:39 likewise returns Failure on 'could not open video'. The wf294 fix widened the return type to Union[List, Failure] and converted the two DETECTOR paths, but left these config-abort paths as bare [], so they remain inconsistent with the codebase's own established Failure convention and still leak as status=\"success\" through _serialize_and_push_outputs. A cloud run submitted before the WASB env has a key set reproduces the wf294 shape verbatim.",
    "failure_scenario": "A wasb_hybrid/tracknet_hybrid/fusion_hybrid cloud job runs with skip_ai_handoff=False but GEMINI_API_KEY unset (or an unknown sport/provider in config). process_video returns bare [] at line 219 (or 194/226). cmd_infer's isinstance(result, Failure) is False, so status is set to 'success', _serialize_and_push_outputs writes report.json status=\"success\"/segment_count=0, and the done-marker carries rc=0/status=\"success\". orchestration.cached_process_result (line 326) and probe_process_cache (line 280) accept the report as a usable success, and _resume_one_remote_job (job_worker.py:462-472) marks the job DONE with 0 rallies ? a misconfig indistinguishable from a legitimately empty match, the exact incident.",
    "suggested_fix": "Return a Failure (is_recoverable=True) from the missing-API-key (line 219), unknown-sport (line 194), and unknown-provider (line 226) paths, matching gemini._resolve_provider. Consider also the video_duration<=0 (line 231) and _resolve_runner NotImplementedError (line 244) paths, which are input/env failures rather than empty matches. Only the trajectory-produced-but-zero-windows return (line 371) is a genuine legit-empty [].",
    "verifier_reasoning": "Confirmed against the real code. trajectory_hybrid.py:219 (missing ${PROVIDER}_API_KEY), :194 (unknown sport KeyError), and :226 (unknown provider KeyError) all still `return []` ? verified in the file and in wf294_fix.diff, which converted ONLY the healthcheck (line 278) and no-trajectory (line 296) paths to Failure while leaving the config-abort paths as bare []. The worker at backend/worker/__main__.py:326 branches on
```

`isinstance(result, Failure)`; a bare [] is not a Failure, so control falls through to line 330 `status = \"success\"`, and line 369 `_serialize_and_push_outputs(... segments, status ...)` writes report.json with status=\"success\"/segment_count=0. Concrete repro: a wasb_hybrid job with skip_ai_handoff=False and GEMINI_API_KEY unset hits line 211 `if not api_key` ? line 219 `return []` ? success/0-segment report ? the exact wf294 shape. The worker's own diagnostic at lines 343-345 explicitly names \"a missing ${AI_PROVIDER}_API_KEY with skip_ai_handoff=False (the segmenter bails BEFORE detection)\" as a top cause, and the sibling gemini._resolve_provider (gemini.py:100/116/124) returns Failure for exactly these three conditions with a docstring citing the same silent-0-rally anti-pattern. So the fix is incomplete and inconsistent with the codebase's own established convention on the most acute pre-detection path. Severity high stands: a common, silent misconfig indistinguishable from a legit empty match, directly in the fix's stated scope."
  },
  {
    "dimension": "completeness",
    "severity": "high",
    "verdict": "CONFIRMED",
    "file": "backend/pipeline/segmenters/yolo_hybrid.py",
    "line": 471,
    "summary": "yolo_hybrid ? the DEFAULT segmenter and a parallel worker-dispatched hybrid ? was left entirely untouched and still returns bare [] on every failure path, so it reproduces the wf294 silent-failure shape while the trajectory twin was fixed, creating a yolo-vs-wasb/tracknet inconsistency.",
    "detail": "yolo_hybrid.process_video imports no Failure and returns bare [] on: missing API key / unknown sport / unknown provider (via _resolve_provider returning None ? line 470-471), and invalid video_duration<=0 (line 482). Its return annotation is still List[Dict[str,Any]] (line 465) with no Failure branch anywhere. It is registered (line 560), worker-dispatchable through the same cmd_infer path, and is in fact the configured default_segmenter (backend/config/models.py:181 default_segmenter='yolo_hybrid'). The wf294 fix touched only the trajectory base class; because cmd_infer's isinstance(result, Failure) branch is generic, yolo_hybrid can never trigger it. This is precisely the CODE_MAP 'twin-parity' footgun the task flagged: a same-class silent-failure hole remains in a sibling segmenter, so a default-segmenter run with a missing key still emits status=\"success\"/0-segments.",
    "failure_scenario": "A cloud (or local) job uses the default segmenter yolo_hybrid with GEMINI_API_KEY unset. _resolve_provider logs an error and returns None; process_video returns bare [] at line 471. cmd_infer records status=\"success\", pushes report.json status=\"success\"/segment_count=0 and a done-marker rc=0, and the control plane (cached_process_result / _resume_one_remote_job) treats it as a completed empty match ? the identical wf294 silent failure, in the default code path, left unaddressed.",
    "suggested_fix": "Apply the same Failure contract to yolo_hybrid: have _resolve_provider return a Failure (message + is_recoverable) instead of None for missing-key/unknown-sport/unknown-provider, return a Failure on video_duration<=0, widen the annotation to Union[List, Failure], and add a test mirroring tests/test_worker.py::test_cmd_infer_detector_failure_marks_report_and_marker_failed for yolo_hybrid. Otherwise the default segmenter silently regresses the exact incident the change was meant to fix.",
    "verifier_reasoning": "Every load-bearing claim reproduces in the code as written. yolo_hybrid.py has NO `from backend.results import Failure` (grep returned no matches) and its process_video return annotation is `List[Dict[str, Any]]` (line 465). _resolve_provider (lines 63-106) returns None on unknown sport (line 76), missing `{PROVIDER}_API_KEY` (line 93), and unknown provider (line 101); process_video then returns a bare `[]` at line 471 (provider is None) and at line 482 (video_duration<=0). The segmenter is registered (line 560, HybridYoloSegmenter) and is the configured default: backend/config/models.py:181 `default_segmenter: str = \"yolo_hybrid\"`. The wf294 fix diff (wf294_fix.diff) touches only trajectory_hybrid.py, worker/__main__.py, and tests ? yolo_hybrid.py is entirely untouched. In backend/worker/__main__.py cmd_infer, line 326 branches on `isinstance(result, Failure)` ? `_fail(3)` (status=failed, rc 3, failure report/marker), but a bare `[]` is not a Failure, so it falls through to lines 329-330 (`segments = []`, `status = \"success\"`) and is serialized/pushed as a status=\"success\"/segment_count=0 report ? the exact wf294 silent-failure shape. Because the isinstance check is generic, yolo_hybrid can never trigger it. This is a concrete, reproducible defect: a default-segmenter run with a missing API key emits a false 0-segment success. The only slight imprecision in the finding is the specific env var name (it depends on config.ai_provider, e.g. GEMINI_API_KEY only if that is the configured provider), but the failure holds for whatever provider is configured, so the substance is intact. High severity is appropriate: this is a same-class silent-failure hole in the DEFAULT code path, exactly the

```
twin-parity inconsistency the change set out to fix."
  },
  {
    "dimension": "completeness",
    "severity": "low",
    "verdict": "CONFIRMED",
    "file": "backend/pipeline/segmenters/trajectory_hybrid.py",
    "line": 463,
    "summary": "Phase-3 where the AI provider errors on EVERY candidate window yields an empty
ai_segments and returns []/status=\"success\", so a run that detected rallies but got zero
usable AI boundaries is indistinguishable from a genuinely empty match.",
    "detail": "In the AI-handoff loop, a provider error per window is only logged (line 427-433,
metrics['error']) and the window contributes []. If every window errors (capacity 503s / rate
limits / all clips fail extraction), ai_segments stays empty and process_video falls through
Phase 4 to return [] ? status=\"success\". This is a real 'ran, but we don't actually know' case
that still masquerades as an empty match, but (a) it is pre-existing, (b) it is shared
identically with yolo_hybrid (line 356) and the gemini segmenter (which surface it via loud logs
+ run_signals counters, not a Failure), and (c) fixing it (e.g. failing the run when all handoff
windows errored) is a broader design decision beyond the wf294 detector-timeout scope. Flagging
for awareness, not as a blocker for this change.",
    "failure_scenario": "A trajectory hybrid detects N>0 candidate windows, but the provider
returns metrics['error'] (503/429) for all N handoff clips. ai_segments is empty, process_video
returns [], cmd_infer writes status=\"success\"/0-segments; the incomplete run is served as a
complete empty result. Lower likelihood/scope than the config-abort and yolo gaps above.",
    "suggested_fix": "Out of scope for wf294's minimal fix; if pursued later, track per-window
provider errors and return a Failure (or a Success-with-warnings) when every handed-off window
errored, mirroring the loud run_signals counters gemini already emits (generate_failed_chunks /
oversize_clips_skipped).",
    "verifier_reasoning": "Traced the exact code path. In trajectory_hybrid.py,
process_candidate_window returns [] when clip extraction fails (line 419) or, on a provider
error, only logs a warning (lines 427-433) while the failed provider call yields no segments ?
so the window contributes []. Line 463 (ai_segments.extend(future.result())) means that if EVERY
handoff window errors, ai_segments stays empty. Phase 4 (lines 474-506) then builds
segments_data from an empty ai_segments and returns a bare [] (not a Failure). On the caller
side, backend/worker/__main__.py cmd_infer (lines 326-330) checks isinstance(result, Failure) ?
a bare [] is not a Failure, so status is set to \"success\" and report.json is written as a
0-segment \"success\" (confirmed by test_cmd_infer_zero_segments_is_logged_loudly asserting
status==\"success\"). Thus a run that detected N>0 candidate windows but got all-provider-errors
is indistinguishable from a genuinely empty match ? exactly as claimed.\n\nThe wf294 diff only
converts DETECTOR-level failures (healthcheck not ready, run_predict producing no trajectory)
into Failure objects; it does not touch the per-window AI-provider-error path, so this gap
remains in the code as written. The finding correctly and honestly frames itself as: pre-
existing, shared identically with yolo_hybrid, out-of-scope for the wf294 detector-timeout fix,
and severity=low \"for awareness, not as a blocker.\" I agree with severity=low ? it is a real
but narrow observability gap that the change deliberately does not attempt to fix, and the
finding says so."
  },
  {
    "dimension": "conventions-tests",
    "severity": "low",
    "verdict": "CONFIRMED",
    "file": "tests/test_worker.py",
    "line": 270,
    "summary": "No end-to-end worker test flows a REAL TrajectoryHybridSegmenter Failure through
cmd_infer; the rc-3 test uses a hand-rolled _FailSeg, and the segmenter unit tests stop at the
segmenter return boundary.",
    "detail": "test_cmd_infer_detector_failure_marks_report_and_marker_failed monkeypatches
segmenter_registry.get to return _FailSeg (test_worker.py line 204-215), a stand-in that just
returns a Failure. The real trajectory_hybrid Failure returns are only verified at the segmenter
level (test_tracknet_hybrid.py test_pipeline_healthcheck_failure_aborts /
test_pipeline_no_trajectory_aborts assert isinstance(res, Failure)). No test drives the ACTUAL
TrajectoryHybridSegmenter/WasbHybridSegmenter healthcheck-fail or no-trajectory path through
cmd_infer to confirm the two ends compose. Both halves are individually locked, so this is a
completeness gap rather than an uncovered hole, but a divergence between the real segmenter's
return type and cmd_infer's isinstance(result, Failure) unwrap (worker/__main__.py line 326-329)
```

would slip past both suites.",
    "failure_scenario": "If a future refactor changed TrajectoryHybridSegmenter to return, e.g., a Success-wrapped Failure or a differently-typed sentinel on the abort paths, the segmenter tests (which assert on the segmenter's own return) and the worker test (which injects _FailSeg) would both still pass while cmd_infer silently mis-routed the real failure back to status=\"success\".",
    "suggested_fix": "Add one worker test that registers/monkeypatches the real WasbHybridSegmenter (with its runner/env mocked to force healthcheck-fail or run_predict->None, as test_tracknet_hybrid already does) and asserts cmd_infer returns rc 3 with report.json status=\"failed\".",
    "verifier_reasoning": "The finding is factually accurate as a test-completeness observation, and it is honestly framed as such (not as a live bug). Verified against real code:\n\n1. All worker tests monkeypatch wmain.segmenter_registry.get to return hand-rolled fake classes (_FakeSeg @156, _EmptySeg @235, _FailSeg @282). The rc-3 test (test_cmd_infer_detector_failure_marks_report_and_marker_failed, test_worker.py:270-343) injects _FailSeg (test_worker.py:204-215), a stand-in that just returns Failure. Confirmed.\n\n2. No worker test imports or drives the real TrajectoryHybridSegmenter/WasbHybridSegmenter. grep of test_worker.py shows process_video only on the fake classes and no import of trajectory_hybrid. The real segmenter's Failure returns are asserted only at the segmenter level in test_tracknet_hybrid.py (test_pipeline_healthcheck_failure_aborts / test_pipeline_no_trajectory_aborts, per the diff, assert isinstance(res, Failure)). Confirmed.\n\n3. The composition point exists exactly as described: worker/__main__.py:326-329 does `if isinstance(result, Failure): ... return _fail(3)`. Confirmed.\n\nSo the two ends are individually locked but never exercised together with the real segmenter. This is a real coverage seam.\n\nSeverity remains low and should NOT be escalated: there is NO concrete input/state that produces wrong behavior in the code AS WRITTEN. The real segmenter currently returns Failure and cmd_infer correctly unwraps it, so the halves compose correctly today. The failure_scenario is explicitly hypothetical (\"If a future refactor changed...\"), i.e. a regression-resistance gap, not a live defect. I mark CONFIRMED because the finding's own claim ? that no end-to-end worker test flows a REAL segmenter Failure through cmd_infer ? is verifiably true, and the finding never overstates it as a current bug. The suggested fix (add one worker test wiring the real segmenter with runner/env mocked to force healthcheck-fail or run_predict->None) is valid and low-cost."
  },
  {
    "dimension": "conventions-tests",
    "severity": "low",
    "verdict": "CONFIRMED",
    "file": "tests/test_worker.py",
    "line": 316,
    "summary": "test_cmd_infer_validation_failure_returns_2 exercises the new _fail(2) report-writing path but asserts only rc==2, leaving the newly-added validation-failure report.json (status/segment_count) unlocked by any assertion.",
    "detail": "_fail now writes+pushes a status=\"failed\" report.json for EVERY failure rc, including validation (rc 2) and unknown-segmenter (rc 2) ? worker/__main__.py line 283-294. Only the rc-3 detector-failure test asserts report.json content and the done-marker; the rc-2 validation test (test_worker.py line 316-341) checks nothing about the written report and passes no --done-uri, so neither the report status nor the marker is verified for the validation branch. A regression that wrote status=\"success\" (or skipped the artifact) on validation failure would not be caught.",
    "failure_scenario": "A refactor that mis-passed the status literal (or reordered so validation _fail skipped _serialize_and_push_outputs) would leave a validation-failed video with a status=\"success\" or missing report.json; the suite stays green because the only validation test asserts rc alone.",
    "suggested_fix": "Extend test_cmd_infer_validation_failure_returns_2 to read outputs/<video_id>/report.json and assert status==\"failed\" and segment_count==0 (and, with a --done-uri, marker returncode==2/status==\"failed\"), mirroring the rc-3 test.",
    "verifier_reasoning": "Traced against the real code. In backend/worker/__main__.py, the validation branch at line 312 (return _fail(2)) and the unknown-segmenter branch at line 320 both invoke _fail, which at lines 283-292 unconditionally calls _serialize_and_push_outputs(scratch, out_uri, video_id, [], \"failed\", ...) ? writing+pushing a status=\"failed\", segment_count=0 report.json for every rc, including the rc-2 validation path. This is the newly-added artifact.\n\nIn tests/test_worker.py, test_cmd_infer_validation_failure_returns_2 (lines 316-341) has exactly ONE assertion: `assert rc == 2` (line 341). It never reads outputs/vidF/report.json, asserts nothing about status or

segment_count, and passes no --done-uri (so no marker is verified). By contrast, the rc-3 sibling test_cmd_infer_detector_failure_marks_report_and_marker_failed (lines 306-313) does read report.json and asserts status==\"failed\", segment_count==0, segments==[], plus marker returncode==3/status==\"failed\". No equivalent coverage exists for the rc-2 validation branch.\n\nFailure scenario reproduces as described: a refactor that mis-passed the status literal for the validation _fail path (e.g. \"success\"), or reordered so validation _fail skipped _serialize_and_push_outputs, would leave a validation-failed video with a wrong/missing report.json and the suite would stay green, because the only validation test asserts rc alone.\n\nSeverity confirmed low: this is a test-coverage/discipline gap, not a shipped-behavior defect. The production path is exercised (rc is verified) and the same _fail report-writing mechanism is locked down by the rc-3 test, so only the validation-branch specialization of the artifact is unasserted. The suggested fix (mirror the rc-3 assertions on report.json status/segment_count and optionally a --done-uri marker) is appropriate."
  }
]

2. assistant (2026-07-06T17:31:56.043Z)

I'll verify the key claims against the actual code before synthesizing. Let me read the critical files.

3. user (2026-07-06T17:31:58.167Z)

```
185             # (worker.cmd_infer, main.py, orchestration) branches on ``isinstance(result,
Failure)``;
186             # the bare-list success/empty paths are the documented incremental gap (CODE_MAP
gotchas).
187             label = self.display_label
188             # --- Sport profile + AI provider wiring (identical across segmenters) ---
189             sport_name = self.config.sport
190             try:
191                 sport_profile = sport_registry.get(sport_name)
192             except KeyError as e:
193                 logger.error(str(e))
194                 return []
195
196             idx_cfg = self.config.indexing
197             skip_ai = bool(idx_cfg.skip_ai_handoff)
198             provider_name = self.config.ai_provider
199             if skip_ai:
200                 model_name = "local-noai"
201                 logger.info(
202                     "%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate windows will "
203                     "be emitted as rallies; no %s handoff, no API key needed.",
204                     label,
205                     provider_name,
206                 )
207             else:
208                 model_name = self.config.ai_model
209                 api_key_env_var = f"{provider_name.upper()}_API_KEY"
210                 api_key = os.environ.get(api_key_env_var)
211                 if not api_key:
212                     from backend.pipeline.segmenters._keyhint import api_key_hint
213
214                     logger.error(
215                         f"{api_key_env_var} environment variable is not set! "
216                         f"To run the {provider_name} handoff, set your API key. "
217                         + api_key_hint(api_key_env_var)
218                     )
219                     return []
220                 try:
221                     provider = provider_registry.get(
222                         provider_name, api_key=api_key, model_name=model_name
223                     )
224                 except KeyError as e:
```

```
                    logger.error(str(e))
                    return []

            video_duration = get_video_duration(video_path)
            if video_duration <= 0:
                logger.error("Invalid video duration.")
                return []
            fps, frame_width = self._probe_fps_width(video_path)

            # --- Runner config + windowing thresholds ---
            # _resolve_runner honours the CPU-stub override (RALLY_DETECTOR_IMPL /
            # indexing.detector_impl="stub") so the pipeline runs GPU/WSL-free; otherwise
            # it delegates to the subclass _build_runner (the real WSL/native runner).
            try:
                runner, runner_params = self._resolve_runner(idx_cfg)
            except NotImplementedError as e:
                # e.g. detector_impl="native" for a family without a native runner
(tracknet, or
                # fusion with a tracknet substrate). Refuse cleanly (no rallies) instead of
crashing.
                logger.error("%s: %s", label, e)
                return []

            velocity_thresh = getattr(idx_cfg, self.family).velocity_threshold
            merge_gap = idx_cfg.dead_time_merge_gap
            min_window_duration = idx_cfg.min_action_window_duration
            # Bounded-windowing over-merge fix (W1): 0 = OFF (unchanged). See
IndexingConfig.
            max_window_duration = idx_cfg.max_window_duration
            window_min_split_gap = idx_cfg.window_min_split_gap
            window_hard_cap = idx_cfg.window_hard_cap
            window_inactivity_end = idx_cfg.window_inactivity_end
            inactivity_rally_thresh = idx_cfg.inactivity_rally_thresh
            inactivity_min_density = idx_cfg.inactivity_min_density
            inactivity_temporal_density = idx_cfg.inactivity_temporal_density
            window_blob_fallback = idx_cfg.window_blob_fallback
            window_blob_factor = idx_cfg.window_blob_factor
            handoff_padding = idx_cfg.ai_handoff_padding
            ai_max_workers = idx_cfg.ai_max_workers
            log_candidates = idx_cfg.log_yolo_candidates
            store_candidates = idx_cfg.store_yolo_candidates

            detection_params = {
                "detector": self.name,
                "velocity_thresh": velocity_thresh,
                "merge_gap": merge_gap,
                "min_action_window_duration": min_window_duration,
                **runner_params,
            }

            # --- Phase 1: env healthcheck (fail fast with a clear message) ---
            ok, msg = runner.healthcheck()
            if not ok:
                logger.error("%s detector environment not ready: %s", label, msg)
                # A detector FAILURE, not a 0-rally video ? surface it so the caller marks
the run
                # failed/retryable instead of writing an empty-but-"successful" result.
                return Failure(
                    message=f"{label} detector environment not ready: {msg}",
                    is_recoverable=True,
                )
            logger.info("%s detector env OK: %s", label, msg)

            # --- Phase 2: trajectory -> candidate rally windows ---
            # Trajectory detectors process the whole video in one pass (they carry
```

```
286             # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
287             t_start = time.time()
288             out_dir = os.path.join(output_base(self.config), self.family)
289             csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
290             if not csv_path:
291                 logger.error("%s produced no trajectory; aborting.", label)
292                 # The cold-start / timeout failure mode (e.g. WASB native inference timed
out after
293                 # timeout_sec, or the detector env yielded nothing). This is a DETECTOR
failure ? NOT
294                 # a legitimately empty match ? so return a Failure the worker marks as
failed/retryable
295                 # rather than a silent 0-segment "success" (the wf294 incident).
296                 return Failure(
297                     message=(
298                         f"{label} detector produced no trajectory (timed out or aborted) ? "
299                         "detector failure, not a 0-rally video."
300                     ),
301                     is_recoverable=True,
302                 )
303
304             points = runner.parse_trajectory_csv(csv_path)
```

4. user (2026-07-06T17:31:59.027Z)

```
85              """Resolve the sport profile, provider, and API key (the process_video
preamble).
86
87              Returns (sport_profile, provider) on success, or a ``Failure`` (with a user-
facing
88              message) when the run cannot proceed ? an unknown sport, a MISSING API KEY, or
an
89              unknown provider. The caller propagates the Failure so the job is marked
**failed**
90              with a reason, instead of a silent 0-rally "success" that looks like an empty
match
91              (Finding B ? the loud-failures bar). A genuine "ran, found nothing" is
unaffected.
92              """
93              # Get sport profile config (config is always typed AppConfig ?
_normalize_config()
94              # in base.py converts plain dicts passed by tests at init time)
95              sport_name = self.config.sport
96              try:
97                  sport_profile = sport_registry.get(sport_name)
98              except KeyError as e:
99                  logger.error(str(e))
100                 return Failure(str(e))
101
102             # Get AI provider and model config
103             provider_name = self.config.ai_provider
104
105             # Get appropriate API key based on the provider
106             api_key_env_var = f"{provider_name.upper()}_API_KEY"
107             api_key = os.environ.get(api_key_env_var)
108             if not api_key:
109                 from backend.pipeline.segmenters._keyhint import api_key_hint
110
111                 msg = (
112                     f"{api_key_env_var} is not set ? the {provider_name} segmenter cannot
run. "
113                     f"Set your API key. " + api_key_hint(api_key_env_var)
114                 )
115                 logger.error(msg)
116                 return Failure(msg)
```

```
117
118             try:
119                 provider = provider_registry.get(
120                     provider_name, api_key=api_key, model_name=model_name
121                 )
122             except KeyError as e:
123                 logger.error(str(e))
124                 return Failure(str(e))
125
126             return sport_profile, provider
127
128         def _maybe_trim_video(
129             self, video_id: str, video_path: str
130         ) -> Tuple[str, Callable[[], None]]:
131             """Optionally trim the video to indexing.debug_duration seconds (debug feature).
132
133             Returns (video_to_upload, cleanup_temp_files) where cleanup_temp_files removes
any
134             temp trimmed file created here.
```

5. user (2026-07-06T17:31:59.878Z)

```
60                  "candidate filtering, then an AI provider for high-precision semantic
boundaries."
61              )
62
63          def _resolve_provider(self, video_path: str):
64              """Phase 0: resolve sport profile + AI provider (with API key).
65
66              Returns the configured provider instance, or ``None`` to signal that
67              ``process_video`` should early-return ``[]`` (bad sport, missing API key,
68              or unknown provider). Behaviour is identical to the inlined version.
69              """
70              # Get sport profile config
71              sport_name = self.config.sport
72              try:
73                  sport_registry.get(sport_name)
74              except KeyError as e:
75                  logger.error(str(e))
76                  return None
77
78              # Get AI provider and model config
79              provider_name = self.config.ai_provider
80              model_name = self.config.ai_model
81
82              # Get appropriate API key based on the provider
83              api_key_env_var = f"{provider_name.upper()}_API_KEY"
84              api_key = os.environ.get(api_key_env_var)
85              if not api_key:
86                  from backend.pipeline.segmenters._keyhint import api_key_hint
87
88                  logger.error(
89                      f"{api_key_env_var} environment variable is not set! "
90                      f"To run the {provider_name} segmenter, set your API key. "
91                      + api_key_hint(api_key_env_var)
92                  )
93                  return None
94
95              try:
96                  provider = provider_registry.get(
97                      provider_name, api_key=api_key, model_name=model_name
98                  )
99              except KeyError as e:
100                 logger.error(str(e))
101                 return None
```

```
102
103             logger.info(
104                 f"Initializing Hybrid Segmenter (torchvision + ByteTrack) for video:
{video_path}"
105             )
106             return provider
107
108         def _collect_candidate_windows(
109             self,
```

6. assistant (2026-07-06T17:32:02.417Z)

Both siblings confirmed. Now let me verify the worker `_fail` path and the two test files.

7. user (2026-07-06T17:32:03.912Z)

```
278             # a status="failed" report.json/intervals.csv (0 segments) so the control
plane's bucket
279             # truth (submit-cache probe, restart-recovery) and the alpha-user result surface
see an
280             # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
must never
281             # mask the original failure. THEN the M6 done-marker (real rc + status) so a
remote
282             # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
expires.
283             try:
284                 _serialize_and_push_outputs(
285                     scratch,
286                     args.out_uri,
287                     video_id,
288                     [],
289                     "failed",
290                     storage_cfg,
291                     str(getattr(args, "video_uri", "") or ""),
292                 )
293             except Exception as e:  # never mask the real failure on an artifact-push hiccup
294                 logger.warning("[worker] could not push failure artifacts: %s", e)
295             # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
296             if getattr(args, "done_uri", None):
297                 _write_done_marker(
298                     args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
299                 )
300             return rc
301
302         segments: List[dict] = []
303         segmenter_actual = None
304         segmentation_ran = False
305         status = "failed"
306         try:
307             if failures:
308                 print(
309                     "[worker] validation FAILED: "
310                     + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
311                 )
312                 return _fail(2)
313             seg_name = args.segmenter or config.indexing.default_segmenter
314             segmenter_cls = segmenter_registry.get(seg_name)
315             if not segmenter_cls:
316                 print(
317                     f"[worker] unknown segmenter: {seg_name!r} "
318                     f"(have {list(segmenter_registry.list_available())})"
319                 )
320                 return _fail(2)
321             segmenter_actual = seg_name
```

```
            result = segmenter_cls(db, config).process_video(
                video_id, local_video, max_frames=args.max_frames
            )
            segmentation_ran = True
            if isinstance(result, Failure):
                print(f"[worker] segmenter FAILED: {result.message}")
                return _fail(3)
            segments = result.value if hasattr(result, "value") else result
            status = "success"
            # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
            # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
            # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
            if not segments:
                detector_impl = (
                    os.environ.get("RALLY_DETECTOR_IMPL")
                    or getattr(config.indexing, "detector_impl", None)
                    or "auto"
                )
                logger.warning(
                    "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
                    "The detector substrate yielded no usable trajectories/windows ? check
the detector "
                    "logs above; common causes: a missing ${AI_PROVIDER}_API_KEY with "
                    "skip_ai_handoff=False (the segmenter bails BEFORE detection ? set the
key or "
                    "indexing.skip_ai_handoff=true; cloud-serving now fails this at
startup), native "
                    "runner UNAVAILABLE (weights/repo/WASB_PYTHON env), the WASB env
produced no "
                    "detections, or the input resolution differs from the tuned proxy
resolution. "
                    "Re-run with -v for the native command + frame counts.",
                    video_id,
                    segmenter_actual,
                    detector_impl,
                )
        finally:
            # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
            emit_indexer_report(
                db=db,
                video_id=video_id,
                video_path=local_video,
                config=config,
                val_results=val_results,
                val_context=val_context,
                segmenter_requested=args.segmenter,
                segmenter_actual=segmenter_actual,
                was_reingest=False,
                segmentation_ran=segmentation_ran,
            )

        # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
        pushed = _serialize_and_push_outputs(
            scratch,
            args.out_uri,
            video_id,
            segments,
            status,
            storage_cfg,
            str(getattr(args, "video_uri", "") or ""),
```

```
377         )
```

8. user (2026-07-06T17:32:04.840Z)

```
200         def process_video(self, video_id, path, max_frames=None):
201             return []
202
203
204     class _FailSeg:
205         """A segmenter whose DETECTOR failed/timed out ? returns a Failure (e.g. WASB
produced no
206         trajectory), which is NOT the same as a 0-rally match. This is the wf294 shape."""
207
208         def __init__(self, db, config):
209             pass
210
211         def process_video(self, video_id, path, max_frames=None):
212             return Failure(
213                 message="WASB detector produced no trajectory (timed out or aborted)",
214                 is_recoverable=True,
215             )
216
217
218     def test_setup_logging_levels():
219         import logging
220
221         wmain._setup_logging(False)
222         assert logging.getLogger().level == logging.INFO
223         wmain._setup_logging(True)
224         assert logging.getLogger().level == logging.DEBUG
225
226
227     def test_cmd_infer_zero_segments_is_logged_loudly(tmp_path, monkeypatch, caplog):
228         # The previous silent "0 segment(s)" gave nothing to analyse (the real cold-start
bug).
229         monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidZ")
230         monkeypatch.setattr(
231             wmain.validator_registry,
232             "run_all_with_context",
233             lambda path, cfg: ([_OKResult()], {}),
234         )
235         monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _EmptySeg)
236         monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
237         monkeypatch.setenv("RALLY_DETECTOR_IMPL", "native")
238         video = tmp_path / "in.mp4"
239         video.write_bytes(b"FAKE")
240         out = tmp_path / "bucket"
241
242         # Call cmd_infer directly (not via main(), whose _setup_logging force-reconfigures
handlers and
243         # would detach caplog's). The warning is emitted by cmd_infer regardless of who
configures logging.
244         args = wmain.build_parser().parse_args(
245             [
246                 "infer",
247                 "--video-uri",
248                 str(video),
249                 "--out-uri",
250                 str(out),
251                 "--scratch",
252                 str(tmp_path / "s"),
253                 "--segmenter",
254                 "wasb_hybrid",
255             ]
256         )
```

```
        with caplog.at_level("WARNING", logger="backend.worker"):
            rc = wmain.cmd_infer(args)
        assert rc == 0  # still a clean run, just empty ? but now EXPLAINED
        msgs = " ".join(r.getMessage() for r in caplog.records)
        assert "0 segments" in msgs and "vidZ" in msgs
        assert "detector_impl=native" in msgs  # surfaces the resolved detector
        assert "native runner UNAVAILABLE" in msgs  # points at the likely cause
        # A segmenter that returns a bare [] is a LEGITIMATE 0-rally match ? status stays
"success"
        # (contrast test_cmd_infer_detector_failure_*, where a Failure surfaces as
status="failed").
        report = json.loads((out / "outputs" / "vidZ" / "report.json").read_text())
        assert report["segment_count"] == 0 and report["status"] == "success"


    def test_cmd_infer_detector_failure_marks_report_and_marker_failed(
        tmp_path, monkeypatch
    ):
        """wf294 fix: a segmenter that returns a Failure (detector timeout/abort) must NOT
be written
        as a 0-segment "success". cmd_infer returns rc 3 and writes report.json + the done-
marker with
        status="failed" so the control plane / alpha-user surface can show an error /
retry."""
        monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidX")
        monkeypatch.setattr(
            wmain.validator_registry,
            "run_all_with_context",
            lambda path, cfg: ([_OKResult()], {}),
        )
        monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FailSeg)
        monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
        video = tmp_path / "in.mp4"
        video.write_bytes(b"FAKE")
        out = tmp_path / "bucket"
        done = tmp_path / "bucket" / "_dispatch" / "tok.done"

        rc = wmain.main(
            [
                "infer",
                "--video-uri",
                str(video),
                "--out-uri",
                str(out),
                "--scratch",
                str(tmp_path / "s"),
                "--segmenter",
                "wasb_hybrid",
                "--done-uri",
                str(done),
            ]
        )
        # A detector failure is rc 3 (distinct from validation's rc 2 and a clean rc 0).
        assert rc == 3
        # report.json is written as an explicit FAILURE, not a false empty "success".
        report = json.loads((out / "outputs" / "vidX" / "report.json").read_text())
        assert report["status"] == "failed" and report["segment_count"] == 0
        assert report["segments"] == []
        # ...and the completion marker the cloud dispatcher bucket-polls carries the same
verdict.
        marker = json.loads(done.read_text())
        assert marker["returncode"] == 3 and marker["status"] == "failed"
        assert marker["segment_count"] == 0 and marker["video_id"] == "vidX"


```

```
316     def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):
317         monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidF")
318         monkeypatch.setattr(
319             wmain.validator_registry,
320             "run_all_with_context",
321             lambda path, cfg: ([_FailResult()], {}),
322         )
323         monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
324         video = tmp_path / "in.mp4"
325         video.write_bytes(b"X")
326         cfg = tmp_path / "config.json"
327         cfg.write_text("{}")
328         rc = wmain.main(
329             [
330                 "infer",
331                 "--video-uri",
332                 str(video),
333                 "--out-uri",
334                 str(tmp_path / "o"),
335                 "--config",
336                 str(cfg),
337                 "--scratch",
338                 str(tmp_path / "s"),
339             ]
340         )
341         assert rc == 2
342
343
344     # --- cmd_train: pull labels -> synth trajectories -> manifest -> harness -> push bundle
---
345
346
347     @pytest.mark.parametrize(
348         "fname,expected",
349         [
```

9. assistant (2026-07-06T17:32:17.281Z)

All five findings verified against the actual code. My verification confirms:

1. `trajectory_hybrid.py:194/219/226` ? config-abort paths still `return []` (lines 194, 219, 226 confirmed), while the sibling `gemini._resolve_provider` returns `Failure` for these exact three conditions (gemini.py:100/116/124). CONFIRMED.
2. `yolo_hybrid.py` ? `_resolve_provider` returns `None`, no `Failure` import, is the default segmenter. CONFIRMED.
3. Phase-3 all-windows-error ? empty ai_segments ? bare `[]` ? status="success". CONFIRMED, correctly scoped as pre-existing/out-of-scope low.
4. `test_worker.py:270` rc-3 test uses `_FailSeg` stub, not a real segmenter. CONFIRMED as coverage-seam low.
5. `test_worker.py:316` validation test asserts only `rc == 2`, no report.json assertion. CONFIRMED as coverage-gap low.

Now synthesizing. Findings 1 and 2 are distinct files but the same root class (config-abort silent-failure leak). They should stay separate since they're in different files with different remediation, but I'll rank them adjacently. Findings 3, 4, 5 are correctly low. The decisive question for the verdict: does the fix as-shipped regress, or is it a net improvement with known remaining gaps?

The fix correctly closes the two detector-timeout paths named in the wf294 incident (healthcheck-not-ready, no-trajectory) and does not regress anything. Findings 1 and 2 are the *same class* of silent failure the fix set out to close, on adjacent paths in the same and sibling files ? but they are pre-existing gaps the fix didn't *introduce*. Given the incident's stated scope ("detector failed/timed out"), the exact timeout path is covered. The config-abort leak is a real, high-severity completeness gap directly adjacent to the fix's intent, but committing the current fix does not make anything worse and delivers the core incident closure.

10. user (2026-07-06T17:32:58.538Z)

```
Structured output provided successfully
```